Rangamati Science and Technology University (RMSTU)

Department of Computer Science and Engineering

OBJECT ORIENTED PROGRAMMING LAB — PROJECT REPORT

Blood Finder

A Trusted Blood-Donor Platform for Bangladesh — connecting verified donors and patients in seconds.

Submitted by	Juli Nath (ID: 2401011004) Asma Akter (ID: 2401011021) Mom Chakraborti (ID: 2401011034) CSE 11th Batch
Submitted to	Dhonita Tripura Assistant Professor, Department of CSE, RMSTU
Live site	https://blood-finder-bangladesh.vercel.app
Source code	https://github.com/julinath/blood-finder

Date of submission: 15 June 2026

Abstract

In Bangladesh, finding blood during an emergency still depends on saved phone numbers, scattered Facebook groups and word of mouth — a slow, unreliable process that also exposes donors to spam and opens the door to fraud and middlemen. **Blood Finder** is a web platform that solves this by letting anyone search a pool of **admin-verified donors** by blood group and district in seconds, post urgent needs on a public **emergency board** where nearby donors step forward, and register as a donor — all on a mobile-first, Bangla interface.

The project began as a JavaFX desktop application for our Object Oriented Programming course and was then rebuilt as a production web application using **Next.js 16**, **TypeScript** and **Supabase (PostgreSQL + Auth)**, keeping the same object-oriented domain model (User, Donor, BloodRequest, DonationRecord). Trust and privacy are the core design principles: a donor's phone number is never public, every donation is confirmed by the receiver (not the donor) to prevent fake counts, and the database itself blocks anonymous visitors from ever reading personal data. The result is a live, usable platform that turns the concepts learned in an OOP lab into a real product.

Table of Contents

1. Introduction

Blood cannot be manufactured — it can only come from a donor. Yet when a patient suddenly needs blood, families are left calling around, posting on social media and hoping a matching donor is nearby, free and eligible. Precious time is lost, and the process is easy to exploit.

Blood Finder is our answer: a single, trusted platform that connects patients with verified blood donors across all 64 districts of Bangladesh. Users can search donors by blood group and area, send a request, or post an urgent need on a public board where nearby donors respond with one tap. The interface is mobile-first and primarily in Bangla, because our target users are ordinary people using everyday phones.

This project also has an academic story. It started as a JavaFX + SQLite desktop application built for our Object Oriented Programming (OOP) Lab, using a clean object model and the Model–View–Controller pattern. We then rebuilt the same idea as a production web application so that it could actually be used by people on their phones — carrying the same OOP design into a modern stack. The original desktop version is preserved on the `desktop-app` branch of the repository.

2. Problem Statement

Finding blood quickly in Bangladesh is harder than it should be:

- **No reliable place to search.** A saved phone number may not answer when it matters; area-based Facebook groups respond slowly and posts sink out of sight; local blood clubs keep separate lists that no one else can see.
- **Some people make money from the crisis.** Donor phone numbers are sold, fake "donors" demand payment at the hospital, and middlemen treat an emergency as a business opportunity.
- **No privacy for donors.** Phone numbers get spread across public groups, so fear of spam and unwanted calls keeps many willing donors away.
- **Fear and wrong beliefs.** Myths such as "donating makes you weak" and a lack of clear information stop many healthy people from donating.
- **No way to know if a donor is eligible.** A donor must wait about 90 days between donations, but there is usually no way to know this in advance.

3. Objectives

- Build a searchable pool of **admin-verified** donors filterable by blood group, district and area.
- Provide a public **emergency board** where patients post urgent needs and nearby donors can respond instantly.
- Protect donors with **donor-first privacy** — a donor's number is never shown publicly; the donor chooses whether to respond.
- Enforce the **90-day eligibility rule** consistently across search, profiles and the request form.
- Prevent fraud: record every donation and let **only the receiver** confirm it, so no one can inflate their own donation count.
- Give administrators tools to **approve donors and moderate** abuse reports.
- Deliver a fast, **mobile-first, Bangla** experience.
- Apply object-oriented design principles in a real, deployable product.

4. Existing System and Its Limitations

Today people rely on a mix of informal channels, each with serious gaps:

Current approach	Limitation
Saved phone numbers / personal contacts	The one person may be unavailable, out of town, or recently donated.
Facebook / area groups	Must find and join the right group; posts get buried; responses are slow and unverified.
Local blood-donation clubs	Each keeps its own list; the lists are not connected or searchable.
Paid "donor" middlemen	Charge money, may be fake, and expose families to fraud.
Stand-alone desktop apps	Single machine, single user, no shared data, must be installed — and our users are on mobile.

None of these provide a single, verified, searchable, privacy-respecting and mobile-friendly system — which is exactly the gap Blood Finder fills.

5. Proposed System (Blood Finder)

Blood Finder is one web platform with three main flows, supported by an administration panel and a statistics map.

5.1 Key Features

- **Find Donors —** search verified donors by blood group, district and area; each card shows an eligibility badge and total donation count.
- **Donor Profiles —** public profile with eligibility countdown and donation history; the contact number is shown only to signed-in users.
- **Blood Requests —** a registered user sends a request to a donor; the donor accepts (revealing their number to the requester) and, after donating, the **requester confirms** "রক্ত পেয়েছি" to complete it.
- **Emergency Board —** post an urgent need; nearby donors respond with one tap ("আমি রক্ত দিতে পারবো"); the board auto-filters to the viewer's own blood group and district.
- **Become a Donor —** a single form (pre-filled from the profile) that, after admin approval, makes the donor visible in search.
- **Admin Panel —** approve or un-list donors, handle abuse reports, and oversee users and emergencies.
- **Statistics & Map —** an interactive 64-district heat-map and blood-group distribution.

6. System Requirements

6.1 Functional Requirements

- Users can register and log in with email/password, a mobile number, or Google.
- A user can apply to become a donor; admins approve before the donor appears in search.
- Visitors can search and filter donors and view public profiles.
- Registered users can send, accept/decline, cancel and complete blood requests.
- Users can post emergency requests and offer to donate on them.
- Users can report abuse, no-shows, fake requests or payment demands.
- Admins can approve/un-list donors and resolve reports.
- The system displays live availability and district/blood-group statistics.

6.2 Non-Functional Requirements

- **Security & privacy:** personal data (email, mobile, health) is unreadable by anonymous visitors; every write is re-checked on the server.
- **Performance:** fast first load on weak phones (server-side rendering); the heavy district map is computed on the server.
- **Usability:** mobile-first, Bangla-first, no horizontal scrolling.
- **Reliability & integrity:** donation counts and duplicate requests are guarded by the database.
- **Maintainability:** typed code, one shared domain model, idempotent SQL.

6.3 Hardware & Software

- **For users:** any modern smartphone or computer with a web browser and internet.
- **For development:** Node.js, a code editor, Git, and a Supabase project; deployed on Vercel.

7. System Design

7.1 System Architecture

The application has three layers: a React client in the browser, the Next.js server (Server Components, Server Actions and route protection) running on Vercel, and Supabase (PostgreSQL + Auth) where Row Level Security and column-level grants protect the data. Public reads go straight to the database with a restricted anonymous key; everything that writes data goes through a server action that re-checks the user.

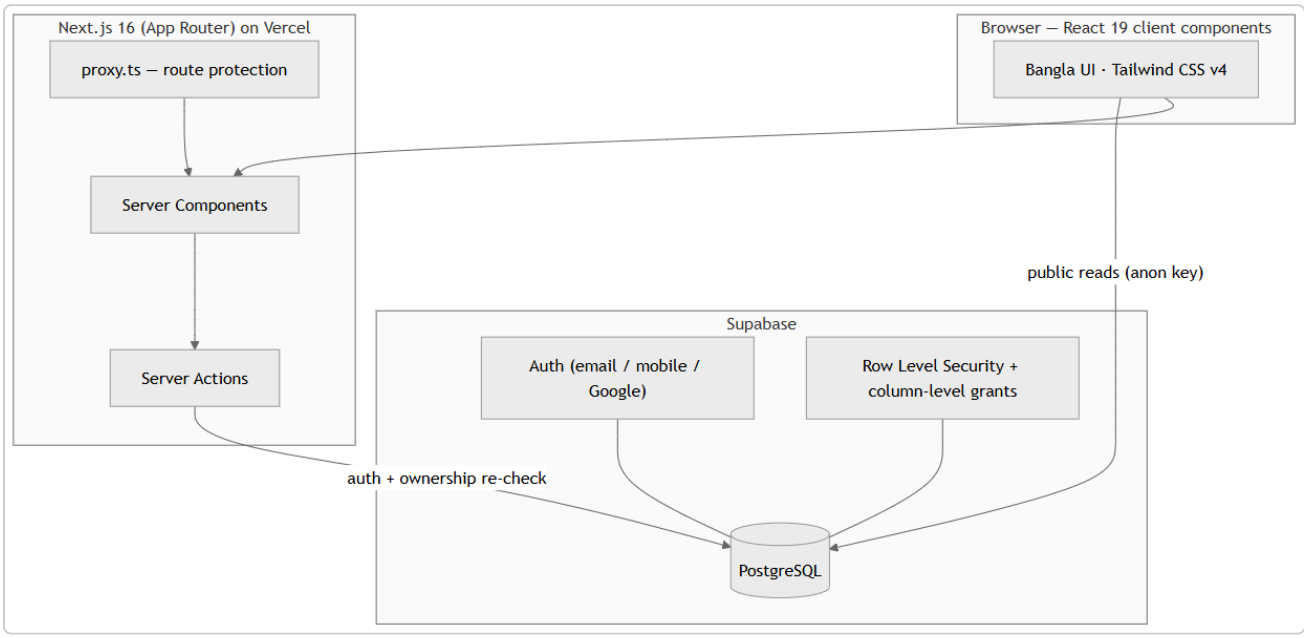

Figure 1: Three-layer system architecture (Client → Next.js → Supabase).

7.2 Use-Case Diagram

Four actors interact with the system: a **Visitor** (not logged in), a **Donor**, a **Requester** (a registered user who needs blood), and an **Admin**.

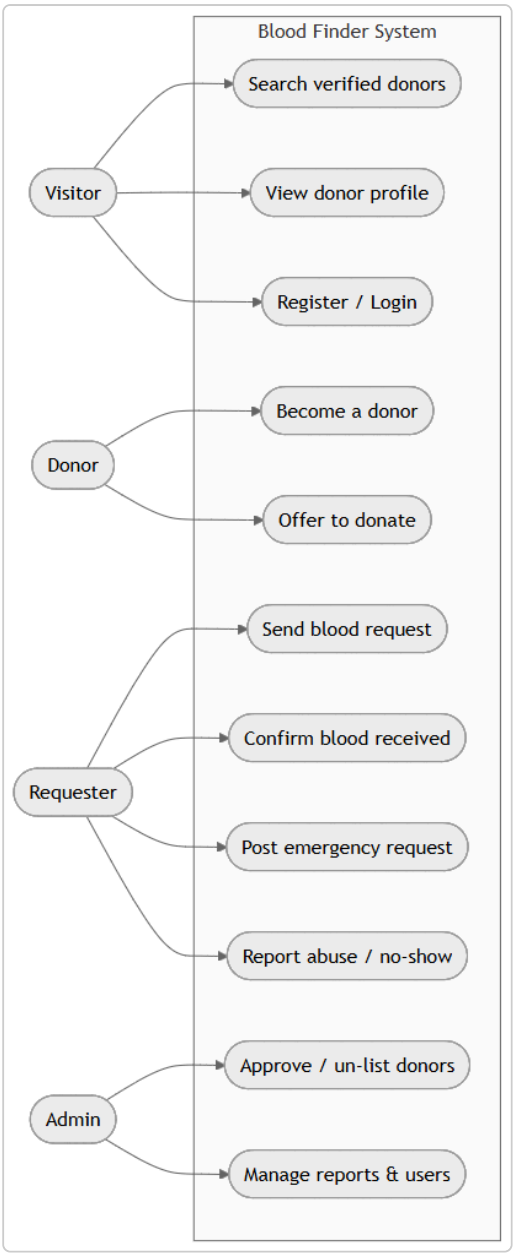

Figure 2: Use-case diagram of Blood Finder.

7.3 Class Diagram

The object model is the heart of the project and is shared between the desktop and web versions. The main classes are `Profile` (user), `Donor`, `BloodRequest`, `DonationRecord`, `EmergencyRequest` and `EmergencyOffer`.

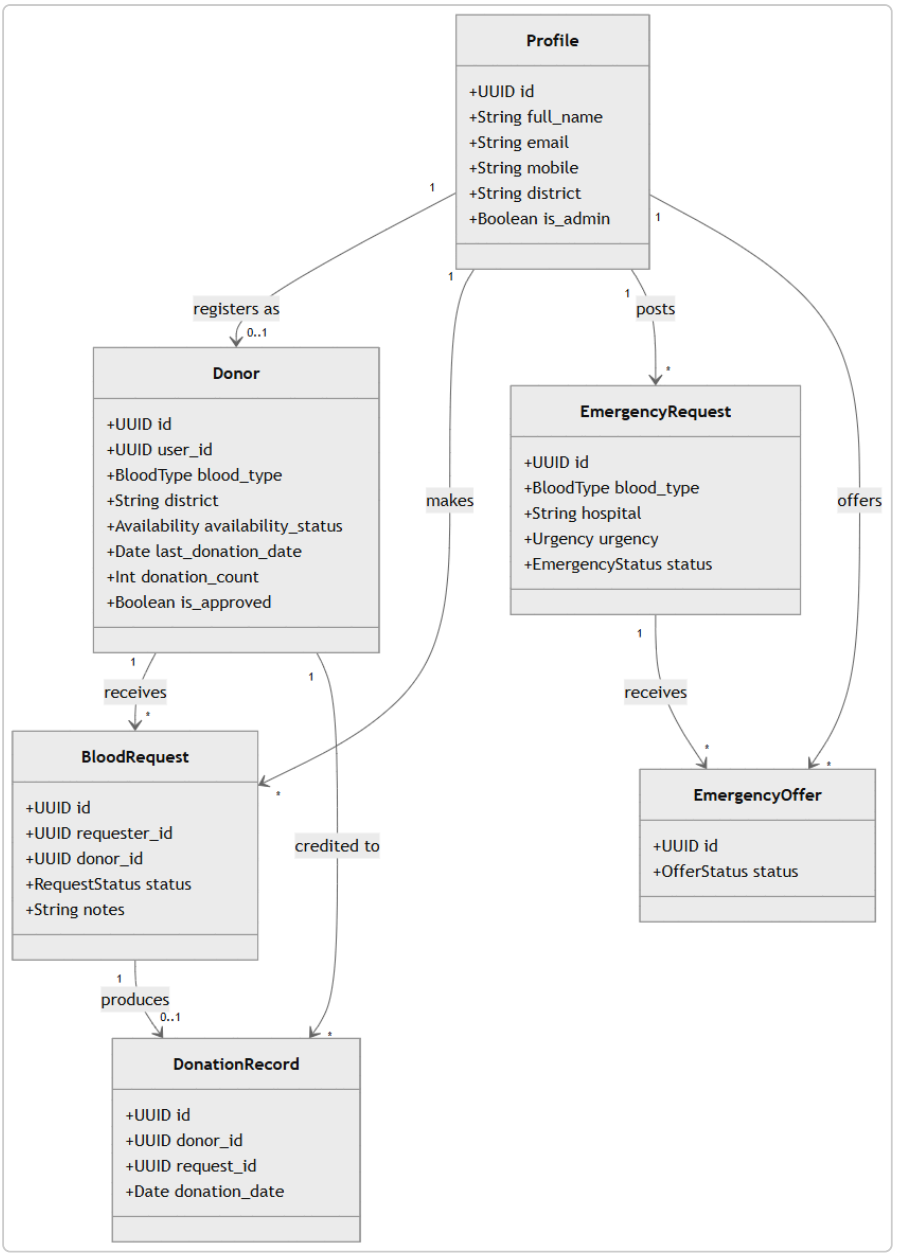

Figure 3: Domain class diagram.

7.4 Database Design

The database has eight tables, all protected by Row Level Security. The entity-relationship diagram below shows how they relate; the table after it summarises each one.

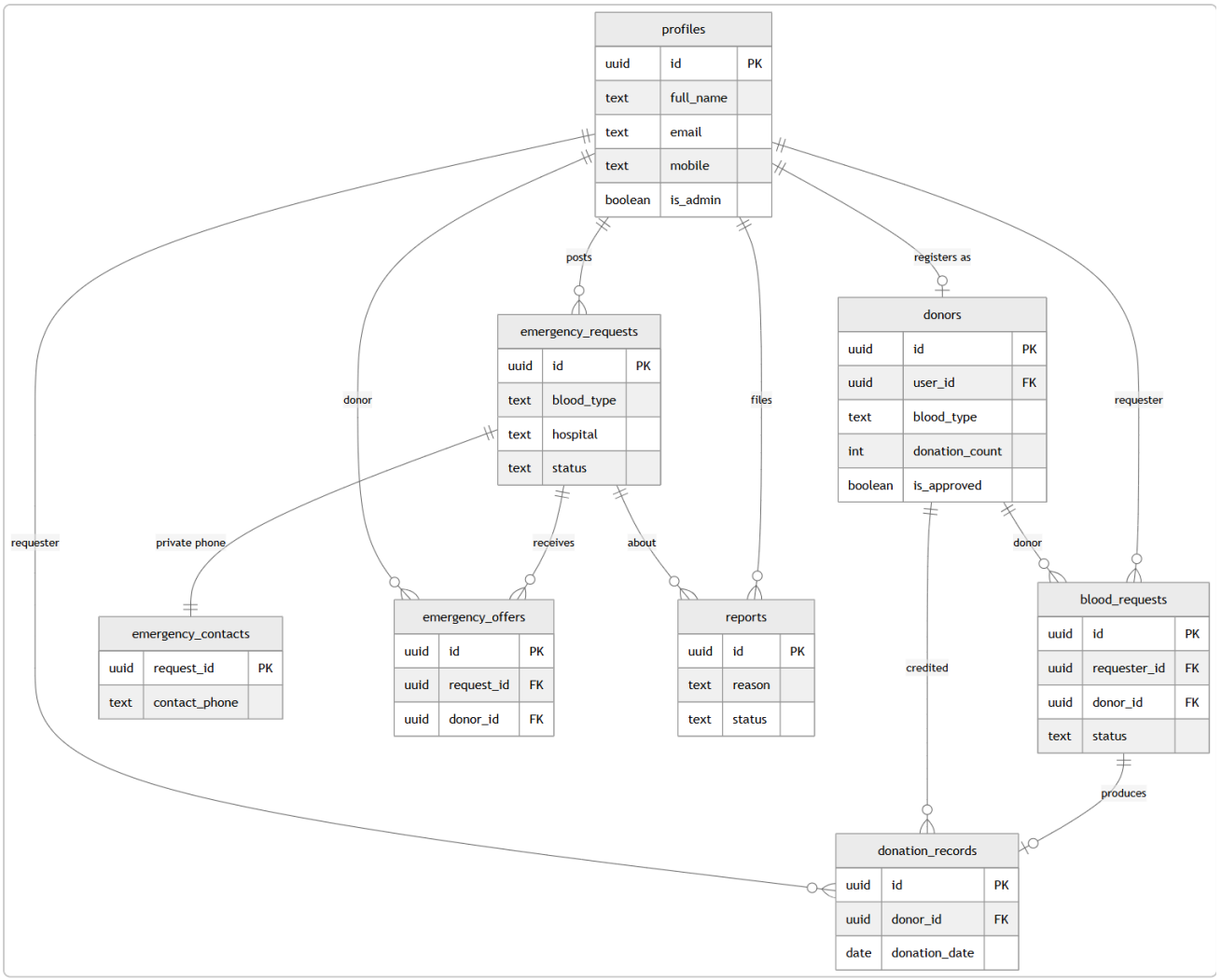

Figure 4: Entity-Relationship (ER) diagram of the database.

Table	Purpose
`profiles`	Extends the auth user — name, email, mobile, district, admin flag.
`donors`	Donor record — blood type, location, availability, last donation date, donation count, approval status.
`blood_requests`	Direct requests from a requester to a donor (PENDING → ACCEPTED → COMPLETED / CANCELLED).
`donation_records`	Immutable log of completed donations; inserting one bumps the donor's count via a trigger.
`emergency_requests`	Public emergency needs board (blood type, hospital, urgency, status).
`emergency_contacts`	The requester's phone number, kept in a separate table so it stays private.
`emergency_offers`	"I can donate" responses from donors to an emergency request.
`reports`	Abuse / no-show / fake-request reports that feed the admin queue.

7.5 Request Lifecycle

The sequence diagram below shows the most important flow — from a request to a recorded donation — and where the anti-fraud rule applies: the donation is confirmed by the **receiver**, never by the donor.

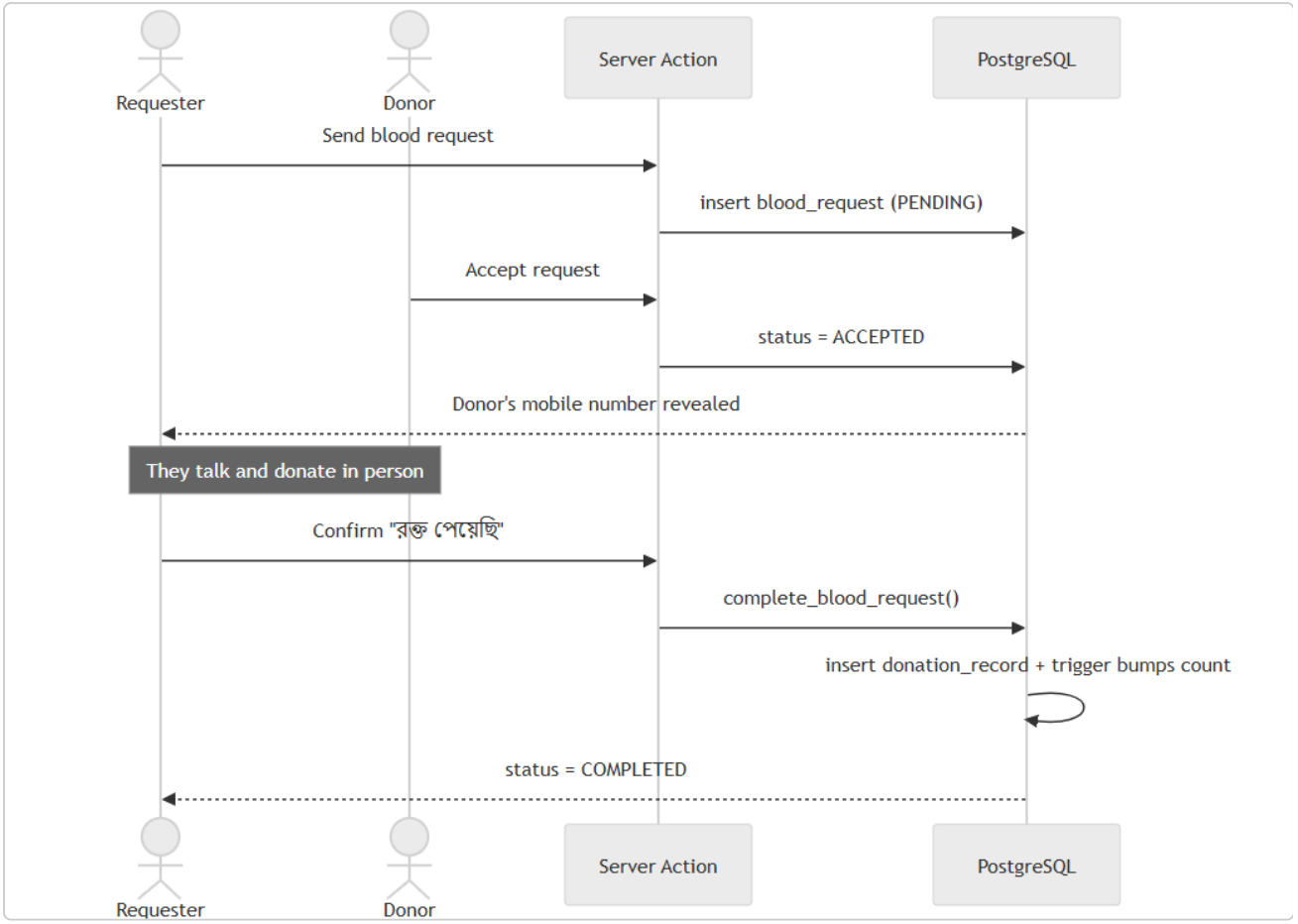

Figure 5: Sequence of a blood request through to a recorded donation.

8. Object-Oriented Concepts Applied

The most important lesson from the OOP course is not a language — it is a way of thinking: split a problem into objects, hide data and details, reuse code, and separate responsibilities. The same domain model we designed for the desktop app lives on in the web app as typed interfaces. The table below maps each concept to both versions.

OOP concept	Desktop (Java / JavaFX)	Web (TypeScript / Next.js)
Encapsulation keep data private	`Donor.java` uses private fields with getters/setters — no other class changes its data directly.	One module (`eligibility.ts`) holds the 90-day rule; the whole site calls that single place.
Abstraction hide the details	A Repository interface hides SQLite; the service layer never sees SQL.	Server Actions hide the database; a UI component just calls `acceptRequest()` .
Inheritance & Polymorphism reuse code	A common base entity and interface implementations — swap the database and the rest stays the same.	Typed unions (`RequestStatus`) and reusable components (`Field` , `BloodTypeBadge`).
MVC / Separation of Concerns split into parts	Model (entities) · View (FXML) · Controller — classic JavaFX MVC.	Database = Model · React pages = View · Server actions = Controller.

> The same model — **User, Donor, BloodRequest, DonationRecord** — that we designed in the OOP lab survives, almost unchanged, as typed interfaces in the web app. That continuity is the project's biggest OOP lesson.

9. Technology Stack

Technology	Role	Why we chose it
Next.js 16 + React 19	Full-stack web framework	UI and server logic in one framework — no separate backend to build and maintain; fast server-side rendering.
TypeScript	Typed language	Our whole domain model is typed, so mistakes are caught while writing code, not after deploy.
Tailwind CSS 4	Styling	Consistent design, tiny CSS output, very fast to build a mobile-first UI.
Supabase	PostgreSQL database + Auth	Real SQL with Row Level Security; email/mobile/Google login ready-made.
Vercel	Hosting + CI/CD	Every `git push` auto-deploys; free, fast and global.
d3-geo	District map projection	The 700 KB map is turned into an image on the server, so it never reaches the phone.
Playwright	Browser testing	Every flow is tested in a real browser, on desktop and 390 px mobile.
Hind Siliguri	Bangla font	Clean, correct Bangla rendering.

10. Implementation

The application is organised into clear modules:

- **Authentication** — email/password, mobile-number sign-up (via a synthetic email) and Google OAuth, handled by Supabase Auth.
- **Route protection** — `proxy.ts` guards the private pages (`/profile`, `/become-donor`, `/request`, `/admin`, `/emergency/new`).
- **Server Actions** — all database writes go through server actions that re-check the logged-in user and their ownership of the data.
- **Business rules** — the 90-day eligibility rule and fitness checks live in one module and are reused everywhere.
- **Atomic operations** — a single database function, `complete_blood_request()`, closes the request and writes the donation record together; a trigger then updates the donation count and the last-donation date.
- **Privacy** — anonymous visitors are given column-level read access only to safe donor-card fields; email, mobile and health data are never exposed.

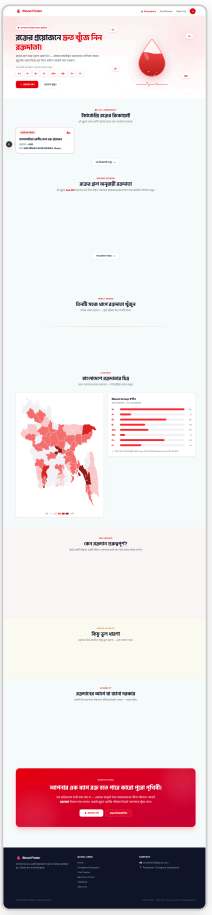

Figure 6: Home page — quick blood-group search, live availability and the district map.

11. Security and Privacy

Trust is the product. Security is applied in three layers, so that a bug in one layer is still caught by the others:

1. **User interface** — input validation and login checks.
2. **Server actions** — every write re-checks authentication and ownership.
3. **Database** — Row Level Security, column-level grants and CHECK constraints. Even a direct API call cannot read a donor's phone number.

11.1 Donor-First Privacy

A donor's phone number is never shown on the public board. When a donor chooses to respond to an emergency, only then does the requester see the donor's number and call them — so the decision always stays with the donor, and there is no spam or harassment.

11.2 Fraud Prevention

> **The receiver confirms the donation — never the donor.** A donor can never mark their own donation as complete, so no one can inflate their donation count. The confirmation is enforced inside the database, and the emergency board follows the same rule.

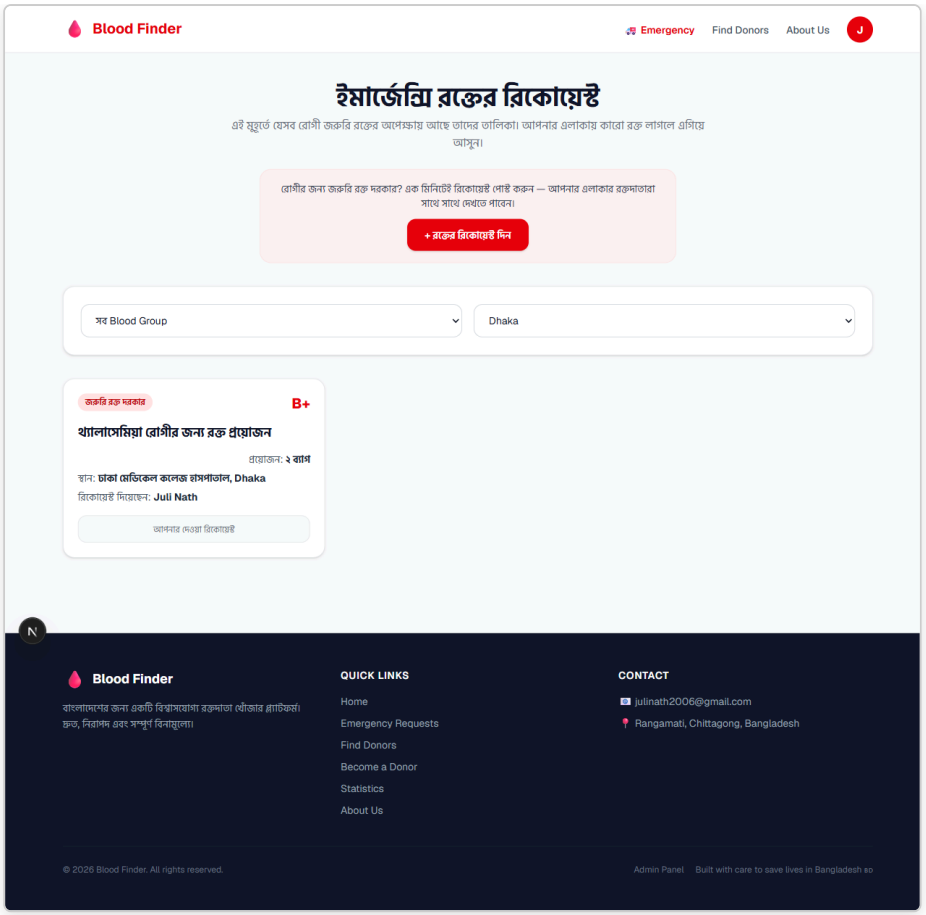

Figure 7: Emergency board — urgent requests with one-tap "I can donate"; the requester's number stays private.

12. Testing

The project was verified at three levels:

- **Type checking** — the TypeScript compiler verifies the whole codebase on every build.
- **Linting** — ESLint enforces a consistent, error-free code style.
- **End-to-end testing** — using Playwright, real user journeys were tested in a real browser (both desktop and 390 px mobile): register → become a donor → admin approval → search → send request → accept → confirm donation → report → admin queue. The production build, type-check and lint all pass cleanly.

Figure 8: Admin panel — donor approval queue, reports queue and oversight.

13. Results (Screenshots)

The platform is fully built and live at **https://blood-finder-bangladesh.vercel.app**. Selected screens are shown below.

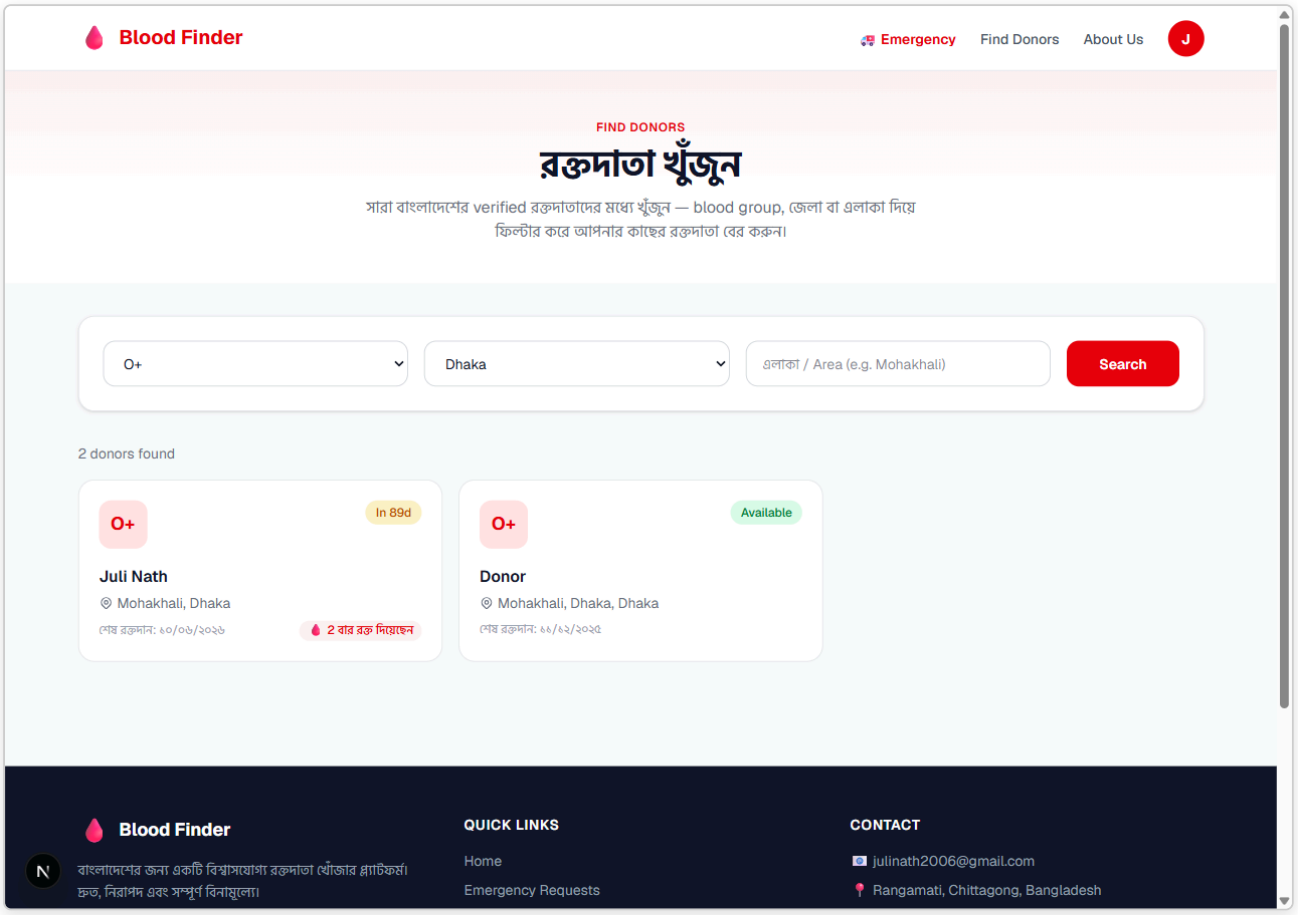

Figure 9: Donor search with blood-group, district and eligibility badges.

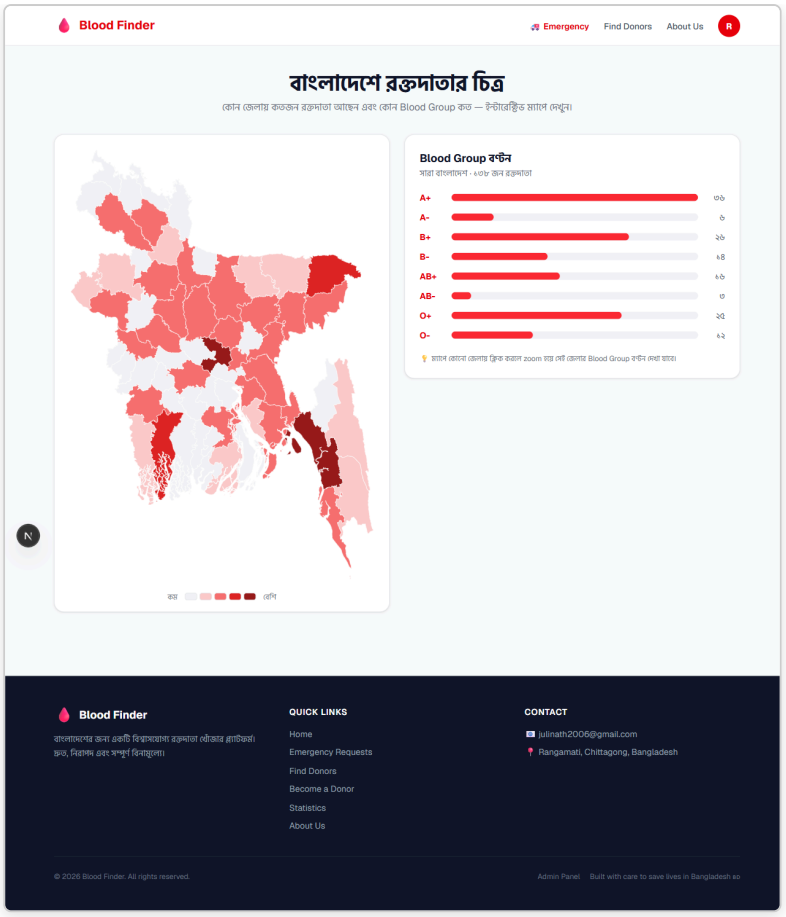

Figure 10: District heat-map and blood-group distribution (rendered on the server).

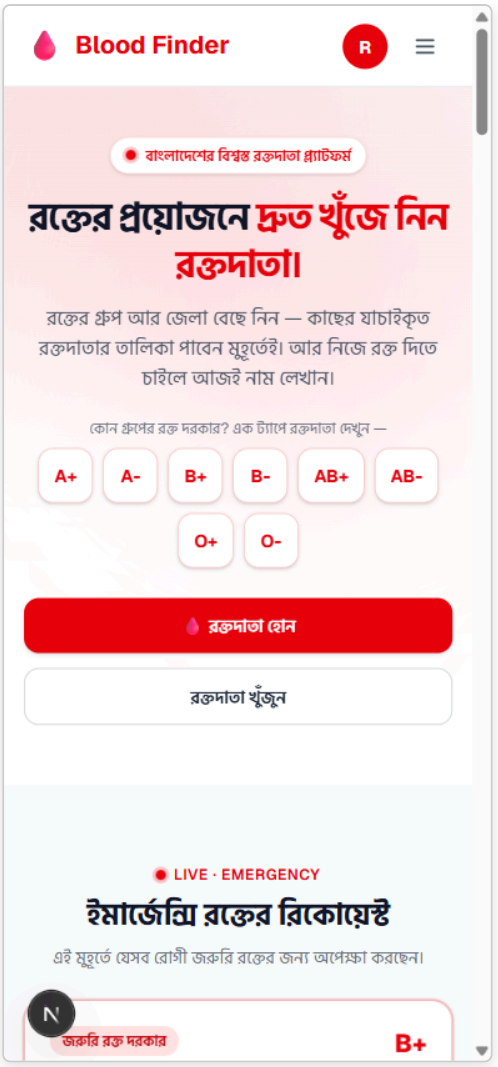

Figure 11: Mobile-first layout — search is visible above the fold at 390 px.

14. Limitations and Future Work

14.1 Limitations

- The platform currently includes demo donor data for demonstration; it grows with real users.
- Notifications are not yet automatic — donors check the board themselves.
- Identity verification is by admin review only (no national-ID check yet).

14.2 Future Work

- **SMS / push notifications** so matching donors are alerted instantly.
- **Donor recognition** — milestone badges (5, 10, 25 donations).
- **Stronger verification** — NID / student-ID checks and hospital partnerships.
- **Smart automation** — auto-expiry of old requests and eligibility reminders.

15. Conclusion

Blood Finder takes a real, painful problem — finding safe blood in an emergency — and solves it with a single trusted platform that is fast, private and free. By removing unreliable middle layers and connecting patients directly with verified donors, it makes the process faster and safer. Just as importantly, it shows how the object-oriented design we learned in the lab — a clean domain model, encapsulation, abstraction and separation of concerns — carries directly into a real, deployable product. The platform is live today and ready to grow into a trusted national blood-donor network.

16. References

1. Next.js Documentation — https://nextjs.org/docs
2. React Documentation — https://react.dev
3. Supabase Documentation (Auth & Row Level Security) — https://supabase.com/docs
4. PostgreSQL Documentation — https://www.postgresql.org/docs
5. Tailwind CSS Documentation — https://tailwindcss.com/docs
6. TypeScript Handbook — https://www.typescriptlang.org/docs
7. Vercel Documentation — https://vercel.com/docs
8. Blood Finder source code — https://github.com/julinath/blood-finder